

API documentation

RAG SaaS Platform

OVERVIEW

API Reference — Overview

The platform exposes a REST API across 89 routers (`api/routers/`). Most resources are scoped under an organization:

```
https://your-instance.example.com/organizations/{org_id}/<resource>
```

A few — notably fine-tuning — use flat paths with `org_id` as a required query parameter instead; see [docs/developer/API.md](#).

Interactive docs

- [Swagger UI](#)
- [Redoc](#)
- [Raw OpenAPI JSON](#) — generated live from the actual

FastAPI app, not hand-maintained.

Reference pages

- [Authentication](#)
- [Errors](#)
- [Rate Limits](#)
- [Pagination](#)
- [Documents](#)
- [Chat](#)
- [Agents](#)
- [Webhooks](#)

Not every one of the 89 routers has its own reference page here — these cover the most commonly integrated-against resources. For anything else, the live [OpenAPI export](#) and [Swagger UI](#) are authoritative and complete.

SDKs

If you'd rather not call the REST API directly, see the [Python](#), [JavaScript](#), [React](#), and [Vue](#) SDKs.

AUTHENTICATION

Authentication

API keys

Send your organization's API key as a bearer token:

```
Authorization: Bearer <your-api-key>
```

Create and manage API keys under your organization's developer settings, or via `api/routers/agent_api_keys.py`. Keys are scoped to the organization they were created in.

Widget keys

The embeddable [widget](#) uses a separate key type (`wgt_...`), scoped for client-side/browser use rather than server-to-server calls — never use an organization API key in a browser context.

Session cookies

The dashboard frontend authenticates members via session cookies set on login (`api/routers/auth.py`), not API keys — this is for the first-party dashboard, not third-party integrations.

Errors

An invalid or missing key returns `401 Unauthorized`. A valid key without permission for the requested resource returns `403 Forbidden`. See [Errors](#) for the full error shape.

CHAT

Chat API

Backed by `api/routers/chat_stream.py` and `api/routers/conversations.py`. See [Chat](#) for end-user behavior.

Conversations and messages are not nested under `/organizations/{org_id}/` in this router — the conversation itself already carries its owning organization, so paths are flat (`/conversations/...`).

Create a conversation

```
POST /conversations
```

Optionally specify `agent_id` to target a specific [agent](#) instead of the default assistant.

Send a message (streaming)

```
POST /conversations/{conversation_id}/messages
```

Response is a streamed sequence of chunks (server-sent events), ending with the final message including citations. Client SDKs handle the stream for you — see [SDKs](#).

Streaming from the public API (`/v1/chat`)

```
POST /v1/chat      header: X-API-Key (scope chat:write)
{ "message": "...", "agent_id": "...", "conversation_id": null, "stream": true }
```

With `"stream": false` (the default) the response is one JSON document. With `"stream": true` it is a `text/event-stream` of `start`, `token`, `citation`, `done` (or `error`) events — the same events as the dashboard's streaming chat. Errors that can be known before generation starts (401/403, 404 for an unknown agent or conversation, 429 rate limit, 402 insufficient credits) are returned as ordinary HTTP errors, not as stream events. The Python and JavaScript SDKs expose this as `chat.stream(...)`.

Editing, retrying, regenerating

```
POST /conversations/{conversation_id}/messages/{message_id}/edit
POST /conversations/{conversation_id}/messages/{message_id}/retry
```

```
POST /conversations/{conversation_id}/messages/{message_id}/regenerate
POST /conversations/{conversation_id}/messages/{message_id}/revert
GET  /conversations/{conversation_id}/messages/{message_id}/edit-history
```

Citations

Each assistant message includes a `citations` array referencing the specific document chunks used to ground the answer — see [Citations](#).

Feedback

Feedback is a separate resource — see `api/routers/feedback.py` — thumbs up/down plus an optional note.

Exporting

```
GET /conversations/{conversation_id}/export/markdown
GET /conversations/{conversation_id}/export/json
GET /conversations/{conversation_id}/export/pdf
GET /conversations/{conversation_id}/export/docx
```

Sharing

```
POST /conversations/{conversation_id}/share
GET  /conversations/{conversation_id}/shares
```

`api/routers/conversation_shares.py` — see [Conversations](#).

Archive, restore, delete

```
POST  /conversations/{conversation_id}/archive
POST  /conversations/{conversation_id}/restore
DELETE /conversations/{conversation_id}
DELETE /conversations/{conversation_id}/permanent
```

DOCUMENTS

Documents API

Backed by `api/routers/documents.py`. See [Documents](#) for the end-user-facing behavior and [Chunking/Embeddings](#) for the processing pipeline.

Upload

```
POST /organizations/{org_id}/documents
Content-Type: multipart/form-data
```

Returns the created document resource with a `status` of `processing`. Processing (ingestion, chunking, embedding) happens in the background — poll the document or listen for a webhook event (see [Webhooks](#)) for completion.

List

```
GET /organizations/{org_id}/documents
```

Paginated — see [Pagination](#).

Get / Delete

Once created, a document is addressed directly by id (not nested under `/organizations/{org_id}/`, same convention as conversations):

```
GET    /documents/{document_id}
DELETE /documents/{document_id}
DELETE /documents/{document_id}/permanent
```

Deleting a document removes its chunks/embeddings from search immediately; it does not retroactively alter citations already present in past generated answers.

Status, versions, and reindexing

```
GET /documents/{document_id}/status
GET /documents/{document_id}/progress
```

```
GET /documents/{document_id}/progress/stream
GET /documents/{document_id}/versions
POST /documents/{document_id}/reindex
```

See also `api/routers/reindex_schedules.py` for scheduled (rather than on-demand) reindexing.

Batch upload

```
POST /organizations/{org_id}/documents/batch
```

See `api/routers/batch_jobs.py` for tracking a batch job's progress.

AGENTS

Agents API

Two distinct resources — see [Autonomous Agents](#) for the architectural difference.

Configured agents (`api/routers/agents.py`)

```
GET /organizations/{org_id}/agents
POST /organizations/{org_id}/agents
```

An agent resource has a system prompt, model config, and an assigned tool set — see [Custom Tools](#). Per-tool permissions are managed separately:

```
GET /organizations/{org_id}/agents/{agent_id}/tools/permissions
POST /organizations/{org_id}/agents/{agent_id}/tools/{tool_name}/permissions
DELETE /organizations/{org_id}/agents/{agent_id}/tools/{tool_name}/permissions/{user_id}
```

Agent run traces (step-by-step execution history) are available under:

```
GET /organizations/{org_id}/agents/runs/{run_id}/traces
GET /organizations/{org_id}/agents/runs/{run_id}/traces/tree
GET /organizations/{org_id}/agents/runs/{run_id}/traces/export
```

Public API

A simpler, versioned surface is also available for external integrations:

```
GET /v1/agents
POST /v1/agents/run
```

Autonomous agents (`api/routers/autonomous_agents.py`)

Creation is organization-scoped; everything else operates on the agent directly by id:

```
GET/POST /organizations/{org_id}/autonomous-agents
GET/PATCH/DELETE /autonomous-agents/{agent_id}

POST /autonomous-agents/{agent_id}/run
```

```
POST /autonomous-agents/{agent_id}/pause
POST /autonomous-agents/{agent_id}/resume
POST /autonomous-agents/{agent_id}/stop
GET  /autonomous-agents/{agent_id}/status

GET  /autonomous-agents/{agent_id}/plans
GET  /autonomous-agents/{agent_id}/plans/{plan_id}
GET  /autonomous-agents/{agent_id}/plans/{plan_id}/steps

GET/POST /autonomous-agents/{agent_id}/memory
DELETE   /autonomous-agents/{agent_id}/memory/{memory_id}

POST /autonomous-agents/{agent_id}/collaborate
GET  /autonomous-agents/{agent_id}/collaborations

GET  /autonomous-agents/{agent_id}/cost
```

Running an autonomous agent creates an `AgentPlan` with `AgentStep`s, executed asynchronously — poll the plan or listen for a webhook event for step-by-step progress. See [docs/autonomous/EXECUTION.md](#) and [docs/autonomous/OVERVIEW.md](#) for the full memory, collaboration, and cost-tracking behavior.

API keys for agent access

Agents/autonomous agents accessed programmatically (rather than through the dashboard) authenticate the same way as any other API consumer — see [Authentication](#) and `api/routers/agent_api_keys.py`.

WEBHOOKS

Webhooks API

Backed by `api/routers/webhooks.py`. See [Webhooks](#) for the conceptual overview.

Registering a webhook

```
POST /organizations/{org_id}/webhooks
{
  "url": "https://your-app.example.com/hooks/rag-saas",
  "events": ["document.processed", "conversation.created", "agent_run.completed", "fine_tuning_job.status_changed"]
}
```

Delivery payload

```
{
  "event": "document.processed",
  "org_id": "...",
  "resource_id": "...",
  "timestamp": "2026-09-14T12:00:00Z",
  "data": {}
}
```

Verifying signatures

Each delivery includes a signature header computed with your webhook's secret (shown once at creation, then only as a masked value). Verify before trusting the payload — see the specific header name and algorithm in your instance's [Swagger UI](#) under `/webhooks`.

Retries

Non-2xx responses are retried with backoff for a limited window. Check delivery history under **Developer settings** → **Webhooks** if events seem to be missing — see [Troubleshooting](#).

RATE_LIMITS

Rate Limits

Rate limits protect the platform from request bursts, independent of your plan's usage quotas (see [Quotas & Limits](#) for the plan-level caps).

Response on limit exceeded

```
HTTP/1.1 429 Too Many Requests
Retry-After: <seconds>
```

```
{"detail": "Rate limit exceeded"}
```

Handling 429s

Respect `Retry-After` and back off exponentially on repeated 429s. Don't retry in a tight loop — this can extend your own throttling window.

Scope

Rate limits apply per API key. A burst against one endpoint counts against your overall limit, not a per-endpoint limit — spreading calls across different endpoints does not avoid throttling.

Auth endpoints

Login, password reset, and 2FA endpoints have stricter, dedicated rate limits as brute-force/spam protection — see [docs/AUTH_BACKEND_SETUP.md](#) for the exact table.

Geo-adaptive limiting

Some deployments enable geo-adaptive rate limiting (stricter limits from unexpected regions) — this is an admin/deployment-level configuration, not something client code needs to account for differently.

ERRORS

Errors

FastAPI's standard error shape:

```
{
  "detail": "A human-readable description of what went wrong"
}
```

Validation errors (422 Unprocessable Entity) include per-field detail:

```
{
  "detail": [
    {
      "loc": ["body", "email"],
      "msg": "field required",
      "type": "value_error.missing"
    }
  ]
}
```

Common status codes

Code	Meaning
400	Malformed request
401	Missing or invalid credentials
403	Authenticated, but not authorized for this resource
404	Resource not found (or not visible to you — the API does not distinguish, to avoid leaking existence across organizations)
409	Conflict (e.g. duplicate resource)
422	Validation error
429	Rate limited — see Rate Limits
5xx	Server error — retry with backoff; if persistent, check Troubleshooting or your monitoring

Provider-specific errors

Some errors originate from an upstream LLM or fine-tuning provider (Anthropic/OpenAI/Mistral) rather than the platform itself — these are surfaced with the real provider error message where possible, not paraphrased or hidden, e.g. a fine-tuning job's `failure_reason` after it fails at the provider. See [docs/fine-tuning/JOBS.md](#).

SDK error types

Client SDKs wrap non-2xx responses in a typed error (`RagSaasAPIError` / `RagSaasAPIError`) carrying the status code and `detail` — see [SDKs](#).